深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

SUAT

数字逻辑 Digital Logic

课程编码： FCAA213 | 开课单位：计算机科学与人工智能学院

开课时间：2026-2027学年秋季学期

主讲教师：马懋德教授 与 李嘉龙助理教授



深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

目录 Contents

Chapter 1 Introduction
Chapter 2 Number & Arithmetic
Chapter 3 Logic Functions
Chapter 4 Verilog Language
Chapter 5 Combinational Circuits
Chapter 6 Basic Storage Elements
Chapter 7 Synchronous Sequential Circuits
Chapter 8 Asynchronous Sequential Circuits
Chapter 9 Digital System Design

深圳理工大学
SHENZHEN UNIVERSITY OF ADVANCED TECHNOLOGY

Chapter 6 Basic Storage Elements

1. Latches

1.1 SR latch

1.2 D latch

2. Flip-flops

2.1 D flip-flop

2.2 T flip-flop

2.3 J-K flip-flop

3. Registers

3.1 N-bit register

3.2 Shift register

4. Counters

4.1 Synchronous Counters

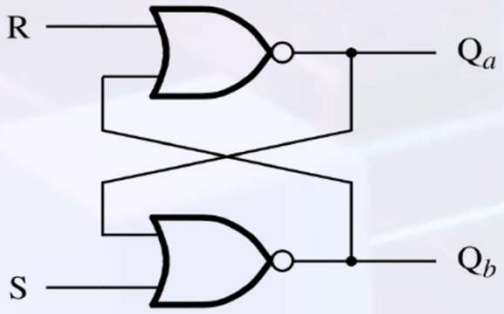
4.2 Ring Counters

5. Verilog Description

基本存储电路

SR Latch

- Two **NOR** gates connected output-to-input
- S (Set 置位) and R (Reset 复位) are added to change the state



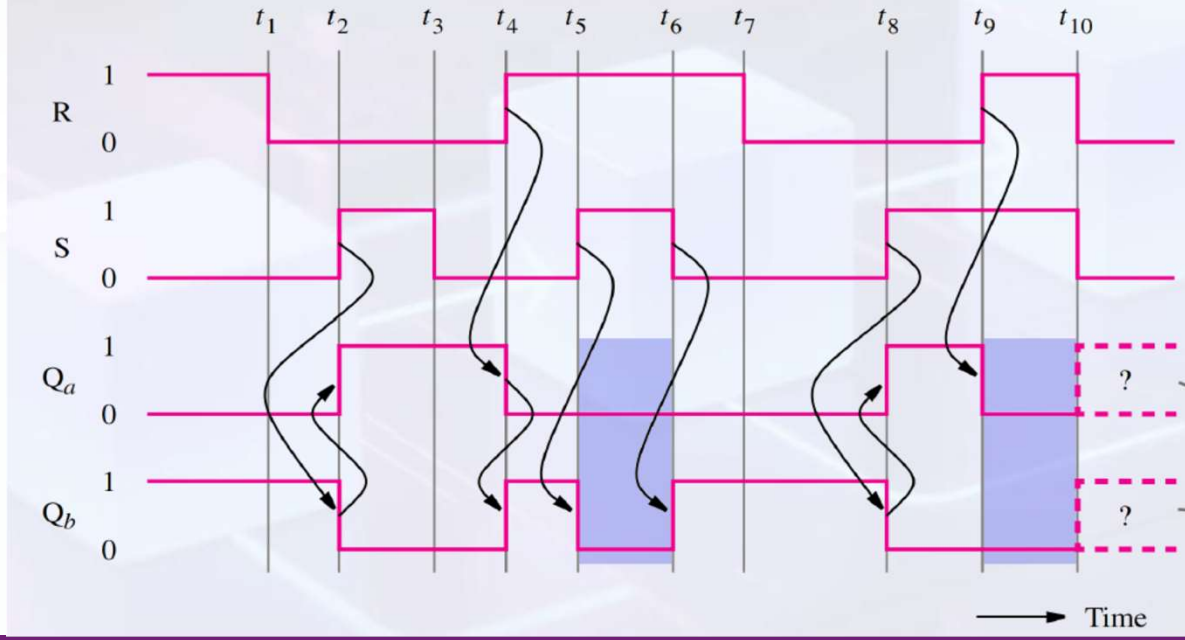
S	R	Q_a	Q_b	
0	0	0/1	1/0	(no change) → Retain previous state
0	1	0	1	**Reset** Q_a to 0
1	0	1	0	**Set** Q_a to 1
1	1	0	0	$Q_a = Q_b = 0$

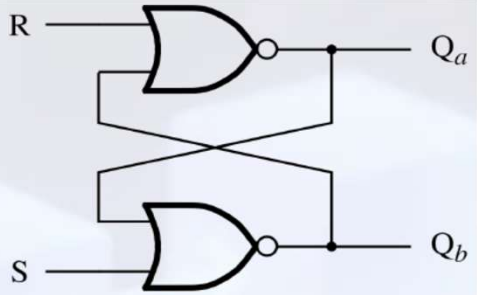
$Q_a = \bar{Q}_b$ $\quad$ $Q_a \neq \bar{Q}_b$

基本存储电路

SR Latch

- **Issue**: the state S=R=1 causes uncertainty





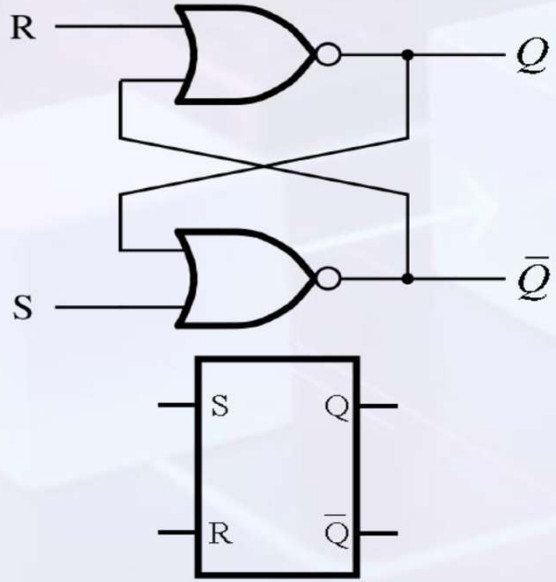
S	R	Q_a	Q_b
0	0	0/1	1/0
0	1	0	1
1	0	1	0
1	1	0	0

Oscillation ! 振荡

基本存储电路

SR Latch

- Set the state S=R=1 to be forbidden state (禁止态)
- The rest states all produce two complementing outputs Q and $\bar{Q}$

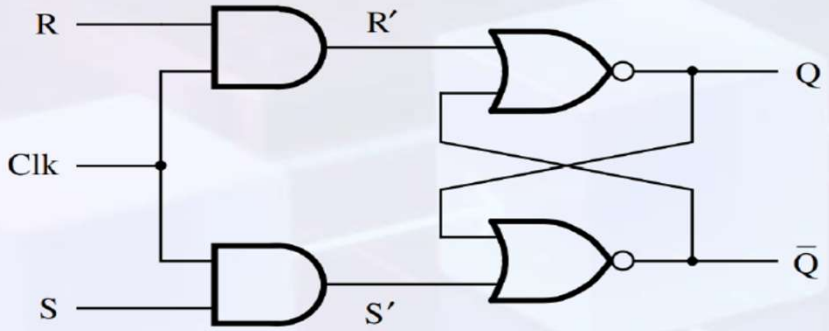


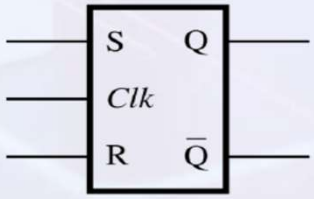
S	R	Q	$\bar{Q}$
0	0	0/1	1/0
0	1	0	1
1	0	1	0
1	1	forbidden	

基本存储电路

Gated SR Latch 门控SR锁存器

- A control input "Clk" is added to enable or disable the entire system
- "S" "R" work only when Clk = 1



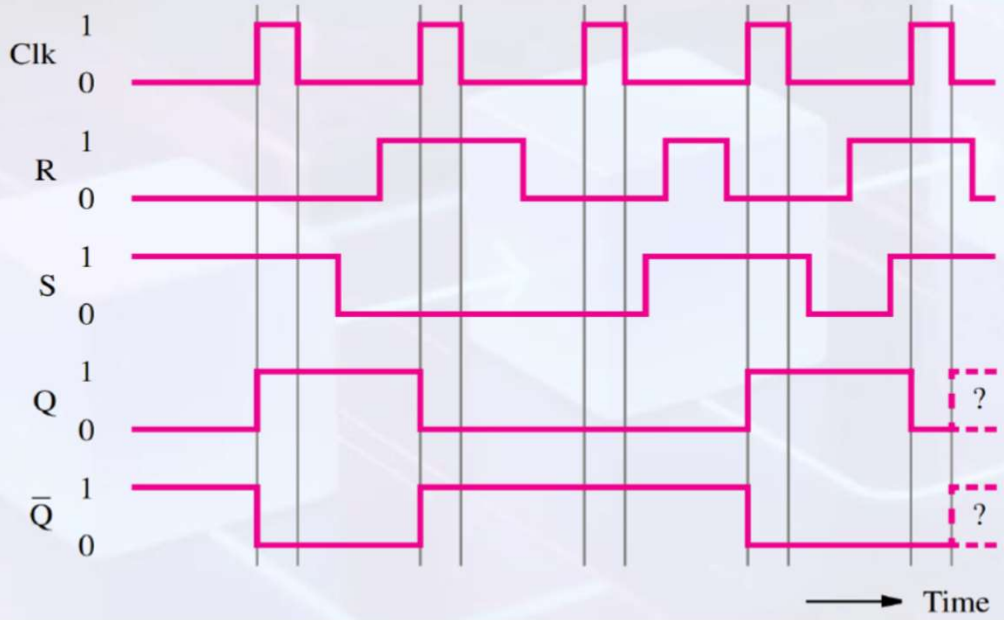


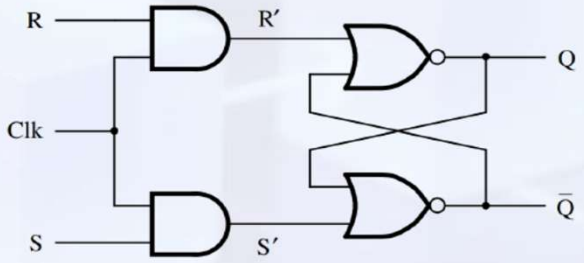
Clk	S	R	Q(*t* + 1)
0	x	x	Q(*t*) (no change)
1	0	0	Q(*t*) (no change)
1	0	1	0
1	1	0	1
1	1	1	x

基本存储电路

Gated SR Latch

- **Issue**: the state S=R=1 causes uncertainty when Clk=0



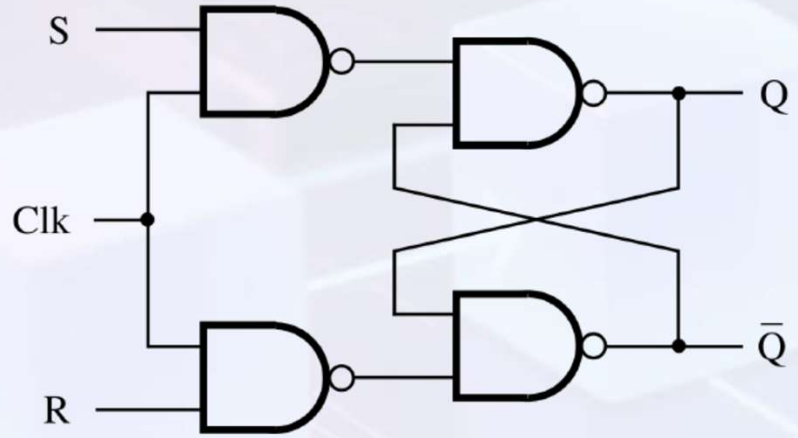


Clk	S	R	Q(t + 1)
0	x	x	Q(t) (no change)
1	0	0	Q(t) (no change)
1	0	1	0
1	1	0	1
1	1	1	x

基本存储电路

Gated SR Latch

- **Gated SR latch with NAND gates**
  - S and R are reversed in comparison with the one with NOR gates

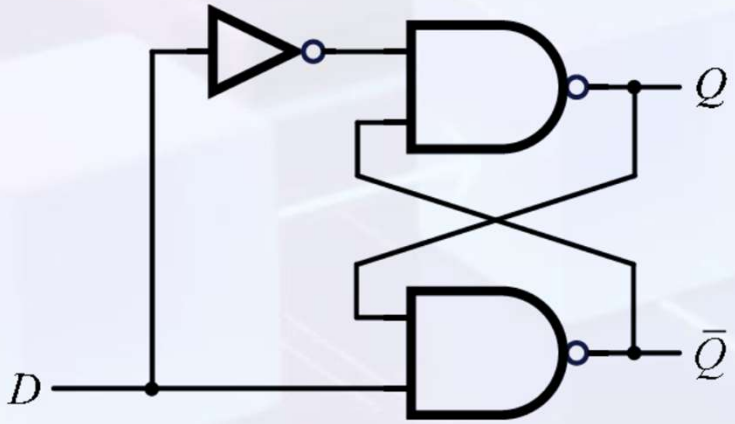


Clk	S	R	Q(t + 1)
0	x	x	Q(t) (no change)
1	0	0	Q(t) (no change)
1	0	1	0
1	1	0	1
1	1	1	x

基本存储电路

D Latch

- A single data input D is used
- No uncertain state anymore



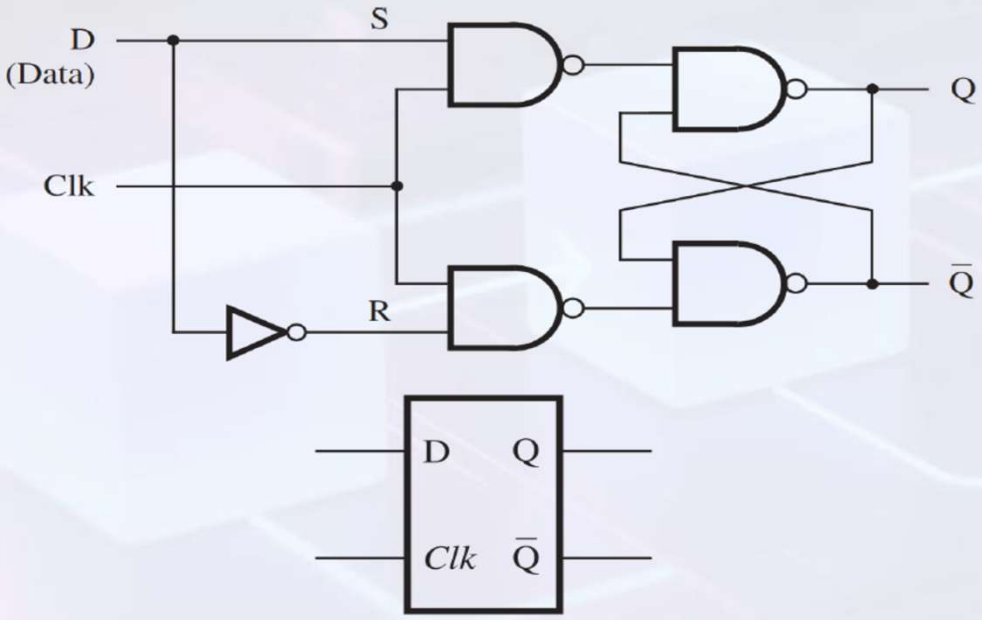
D	Q	$\bar{Q}$
0	0	1
1	1	0

$$Q = D, \quad \bar{Q} = \bar{D}$$

基本存储电路

Gated D Latch

- Output Q follows input D when Clk = 1, otherwise retain previous

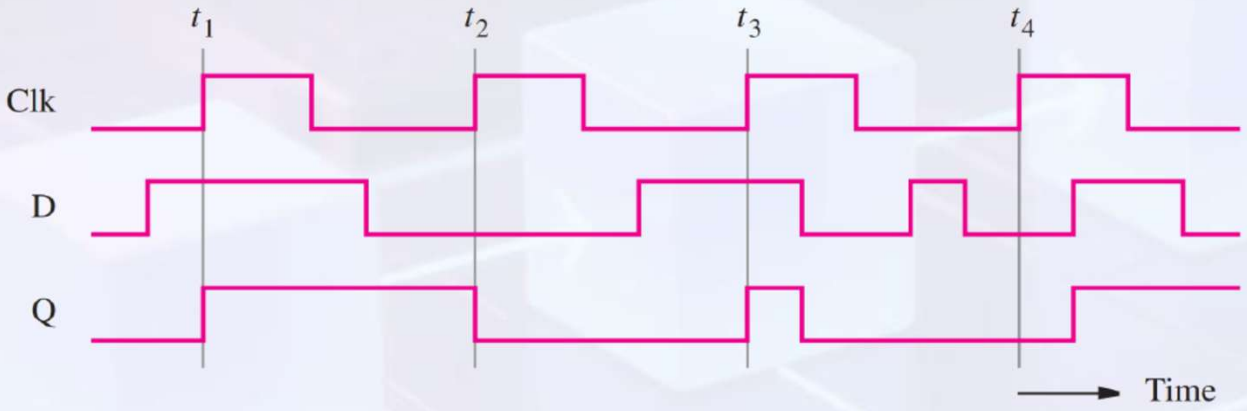


Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

基本存储电路

Gated D Latch

- Output Q follows input D when Clk = 1, otherwise retain previous

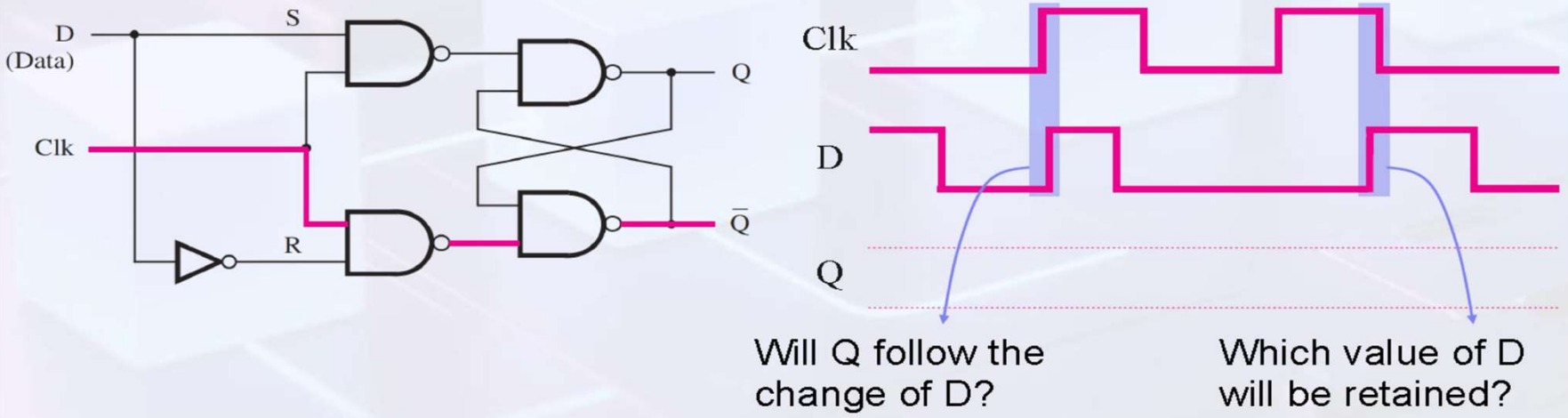


Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

基本存储电路

Propagation Delay

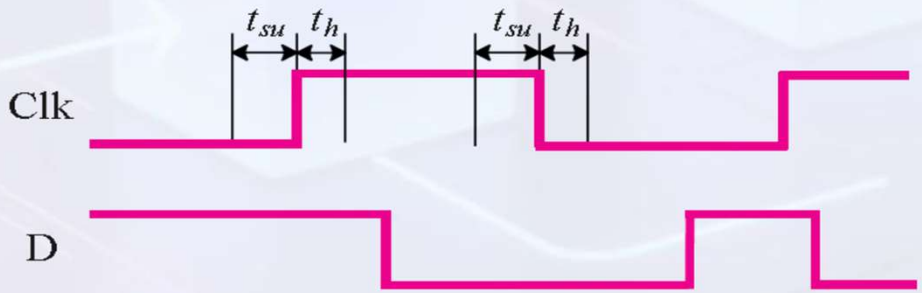
- Delay is not considered so far, but in practical circuits, it is essential to take delays into account
- During the propagation delay from Clk to Q, what if D changes?



基本存储电路

Propagation Delay

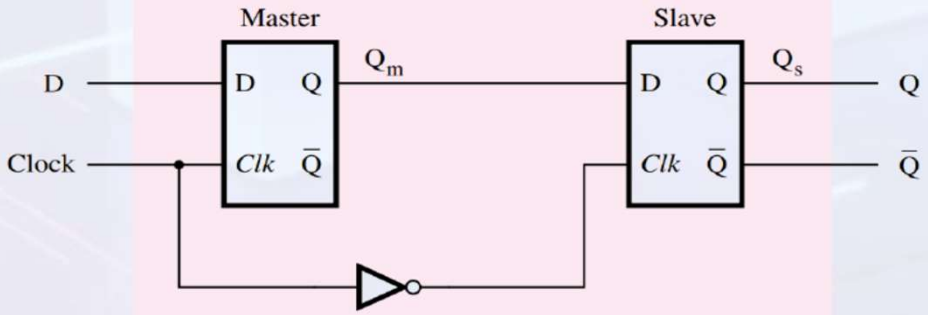
- Two timing constraints are set:
  - t_{su} : setup time 建立时间, the minimum time that the D signal must be stable prior to the edge of the Clk
  - t_h : hold time 保持时间, the minimum time that the D signal must remain stable after the edge of the Clk



基本存储电路

D Flip-Flops D触发器

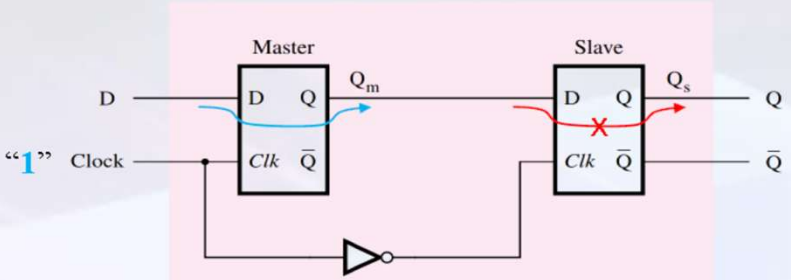
- Sometimes we want storage elements to change states no more than once during one clock cycle
- Master-slave D flip-flop (主从D触发器)
  - Contains two D latches



基本存储电路

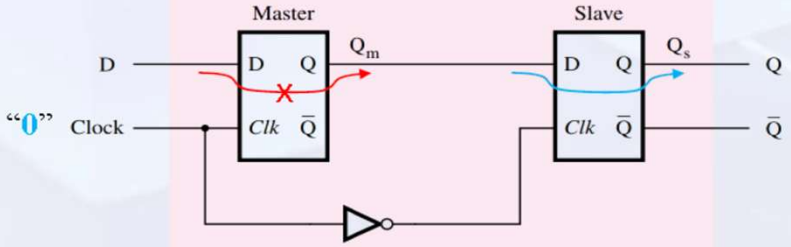
D Flip-Flops

- Master-slave D flip-flop (主从D触发器)



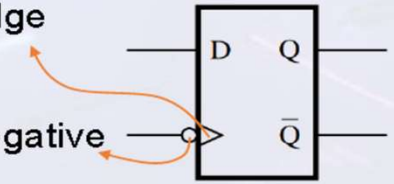
Clock	State
1	$D \rightarrow Q_m$, Q_s retain
0	Q_m hold , $Q_m \rightarrow Q_s$

- A **1**-to-**0** edge ↓ is needed for $D \rightarrow Q$
- Negative edge triggered



"**>**" represents edge sensitive

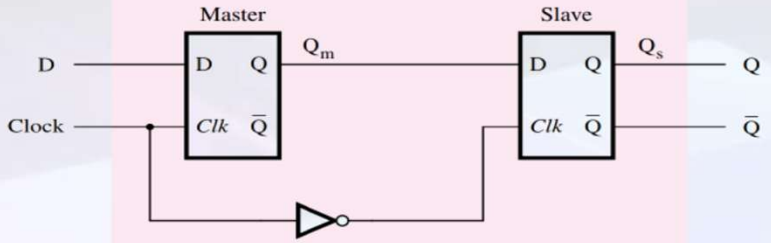
"**o**" represents negative edge sensitive



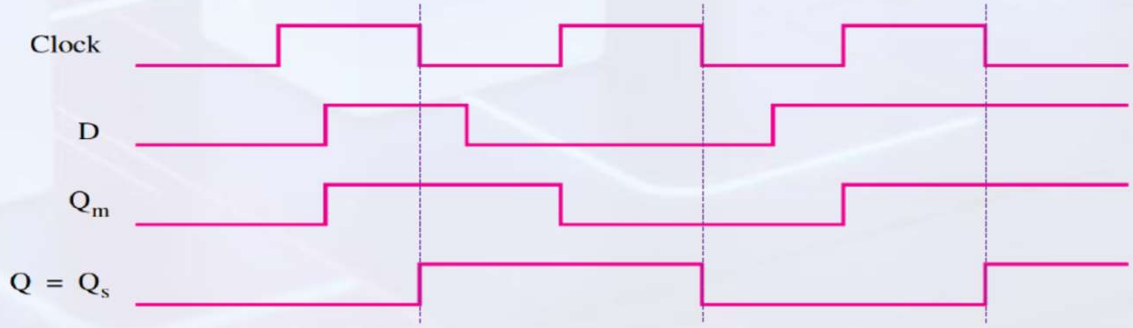
基本存储电路

D Flip-Flops

- Master-slave D flip-flop (主从D触发器)



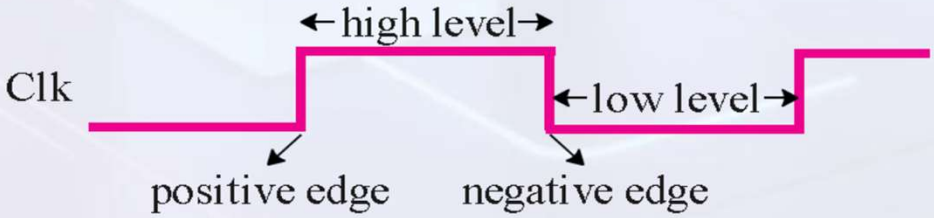
Clock	State
1	$D \rightarrow Q_m$, Q_s retain
0	Q_m hold , $Q_m \rightarrow Q_s$



基本存储电路

Level-sensitive & Edge-sensitive

- Level-sensitive 电平敏感:
  The state changes with input signals during the period when the Clk signal is "**1**", just like in latches
- Edge-sensitive 边沿敏感:
  The state changes when the Clk signal changes from "**0**" to "**1**" (**positive edge** 上升沿), or from "**1**" to "**0**" (**negative edge** 下降沿)



基本存储电路

Practice:

Problem: How to build a positive edge triggered D flip-flop?

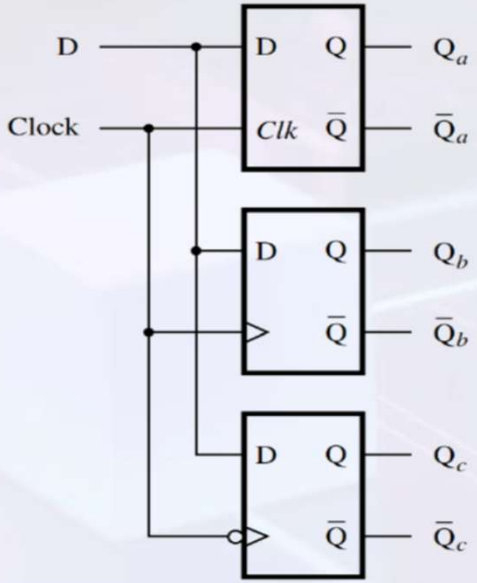
Clock	State
0	$D \rightarrow Q_m$, Q_s retain
1	Q_m hold , $Q_m \rightarrow Q_s$

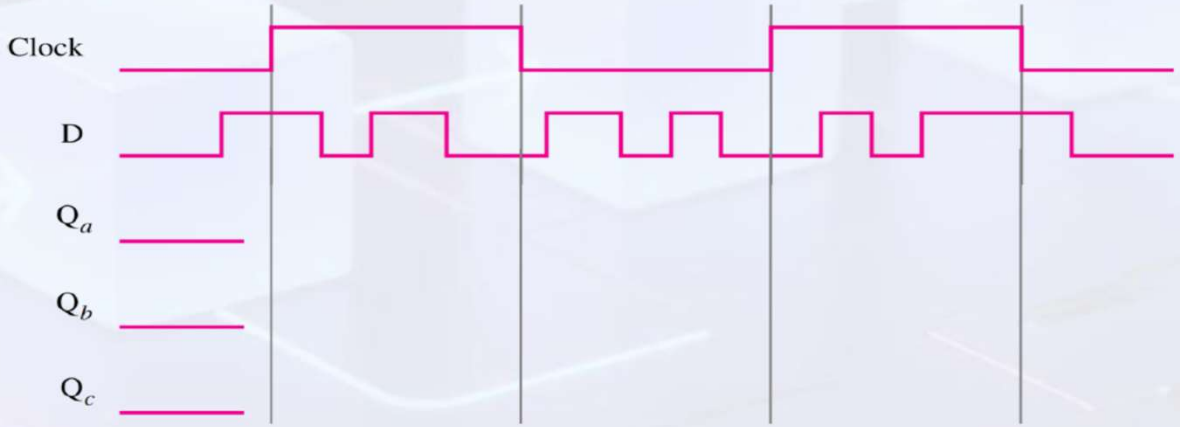


基本存储电路

Practice:

Problem: Comparison of different storage elements.

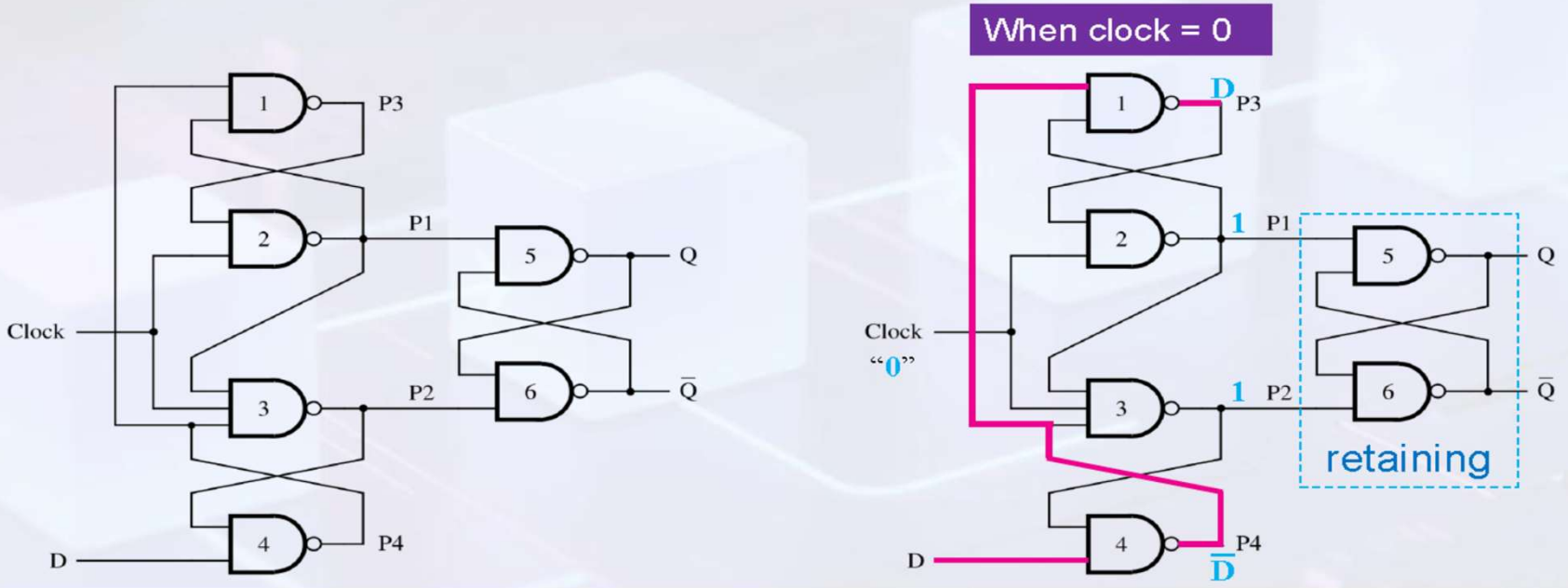




基本存储电路

D Flip-Flops

- Another type of D flip-flop, with 6 NAND gates



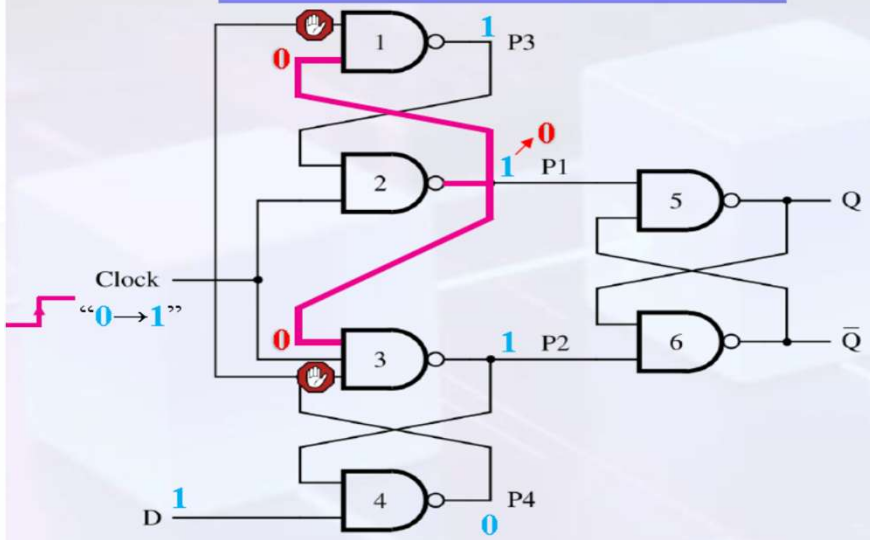
基本存储电路

D Flip-Flops

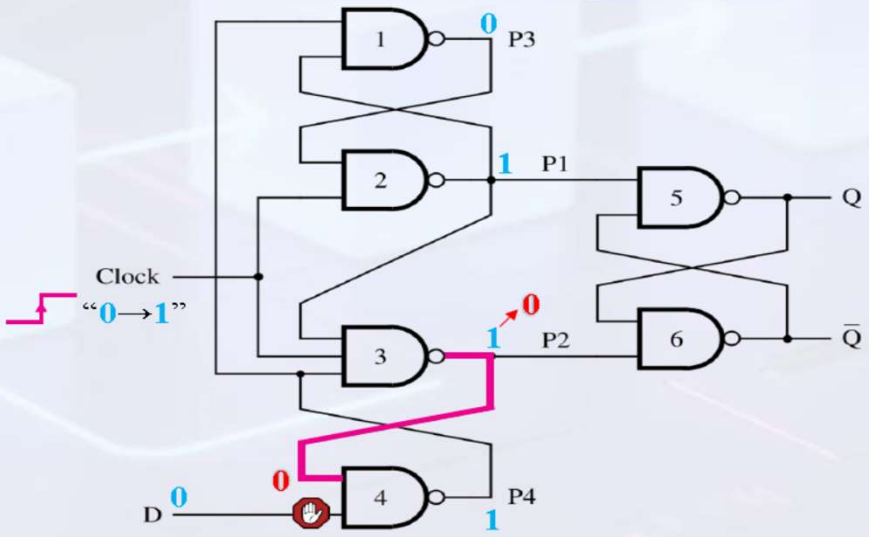
- Another type of D flip-flop

When clock = 1

If D=1 when the edge comes



If D=0 when the edge comes

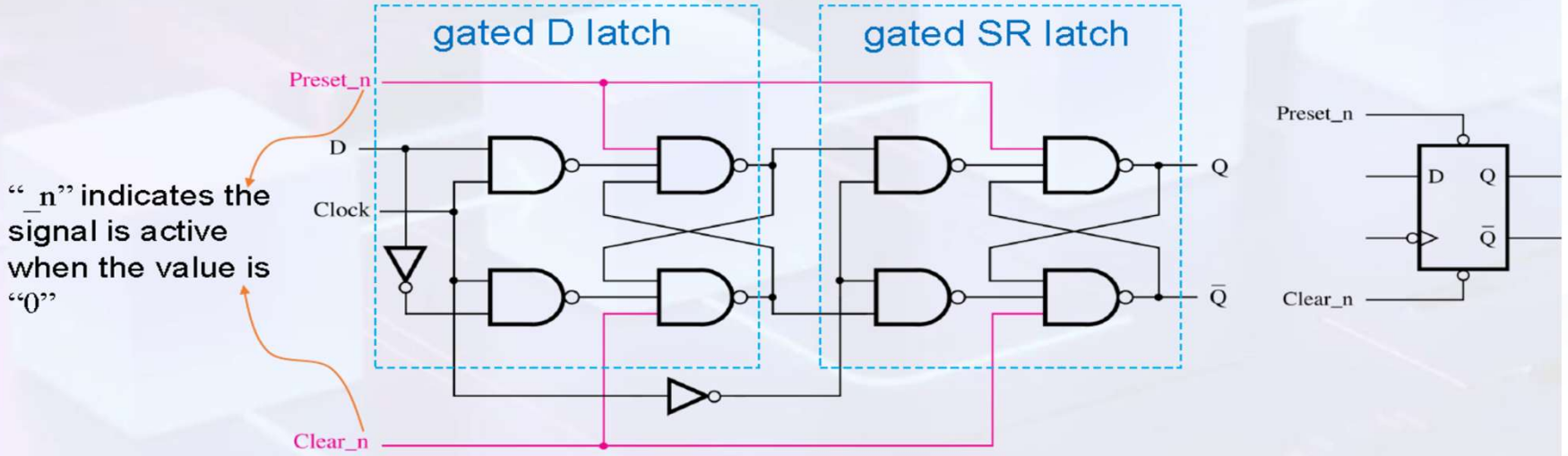


D is blocked when clock=1, its change will not affect the stored states of P1 and P2

基本存储电路

D Flip-Flops with Clear and Preset

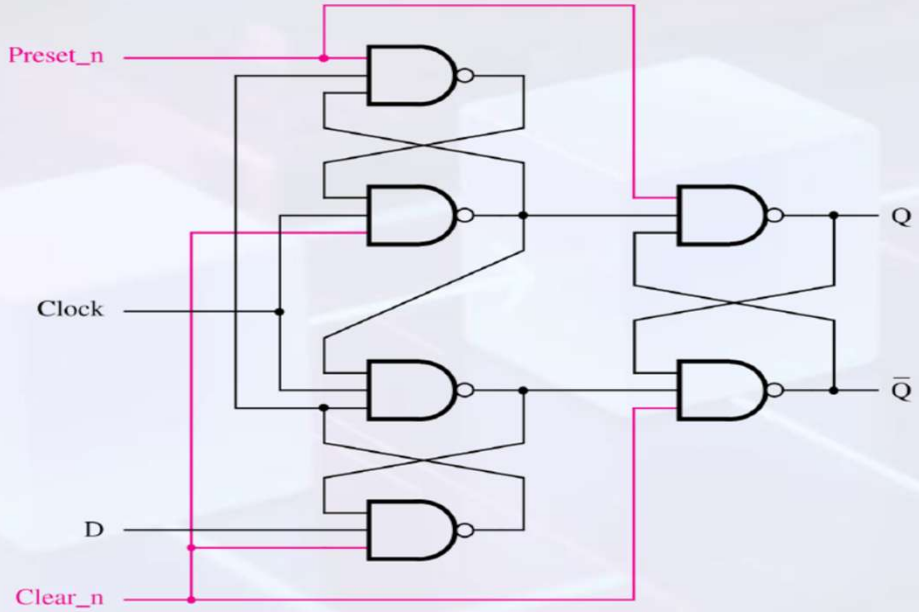
- Clear (清零): clear all flip-flops states to Q = 0
- Preset (预置): set all flip-flops states to Q = 1

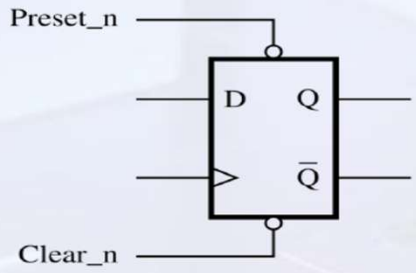


基本存储电路

D Flip-Flops with Clear and Preset

- Another example

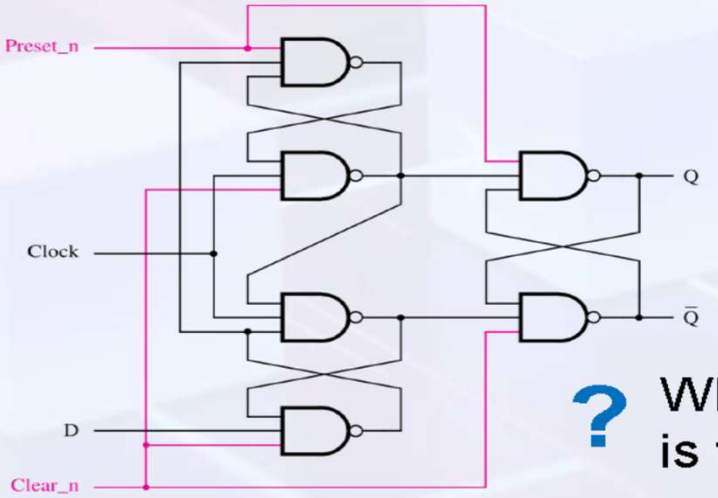




基本存储电路

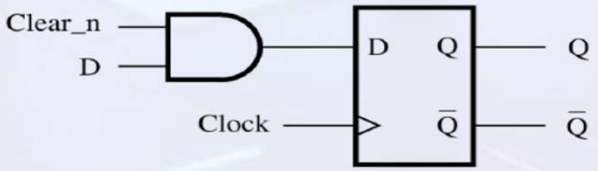
Asynchronous and Synchronous Clear

- Asynchronous clear (异步清零): **immediately**, regardless of the value of the clock
- Synchronous clear (同步清零): on the active **edge** of the clock



? Which kind is this clear?

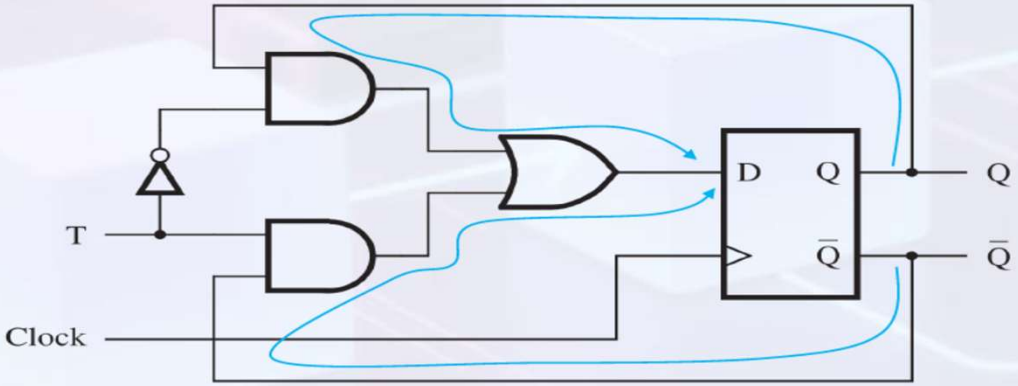
synchronous clear



基本存储电路

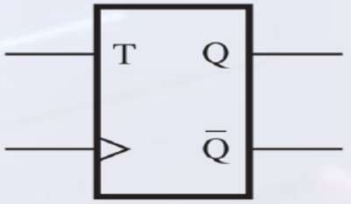
T Flip-Flop T触发器

- T means “toggle 翻转”
- No external input
- Any circuit that implements this function is called a T flip-flop



$$D = T\bar{Q} + \bar{T}Q$$

T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$

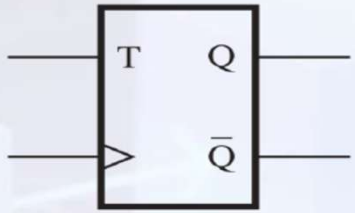


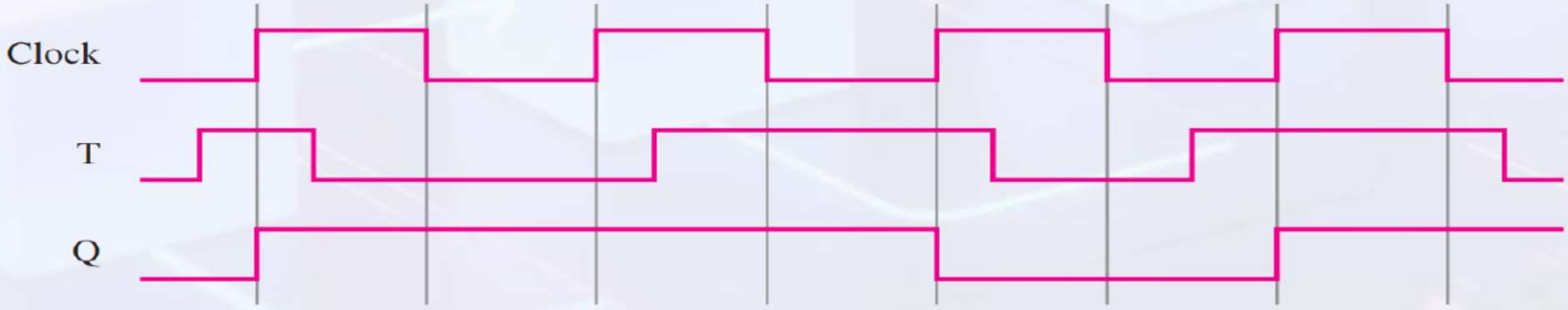
基本存储电路

T Flip-Flop T触发器

- Timing diagram of T flip-flop

T	$Q(t+1)$
0	$Q(t)$
1	$\overline{Q}(t)$

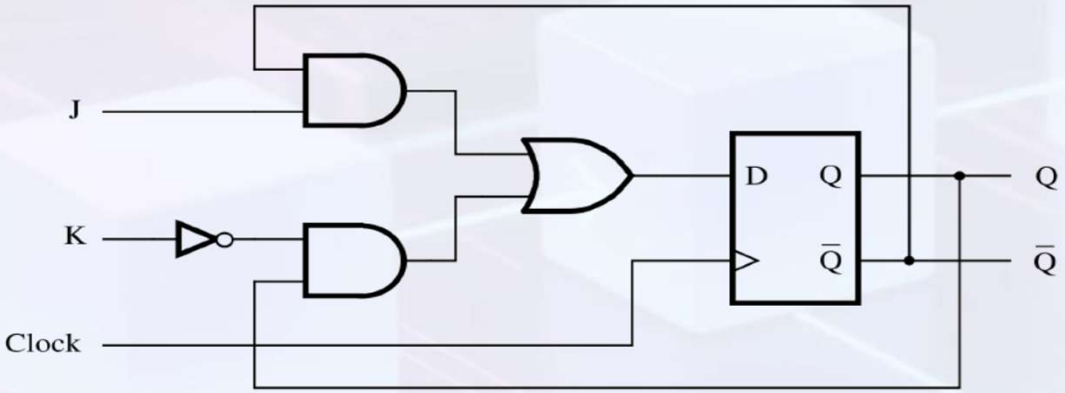




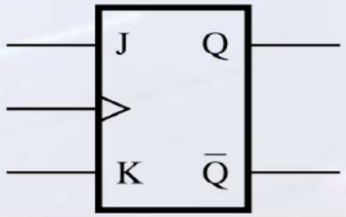
基本存储电路

JK Flip-Flop JK触发器

- Combine the behaviors of SR and T flip-flops in a useful way



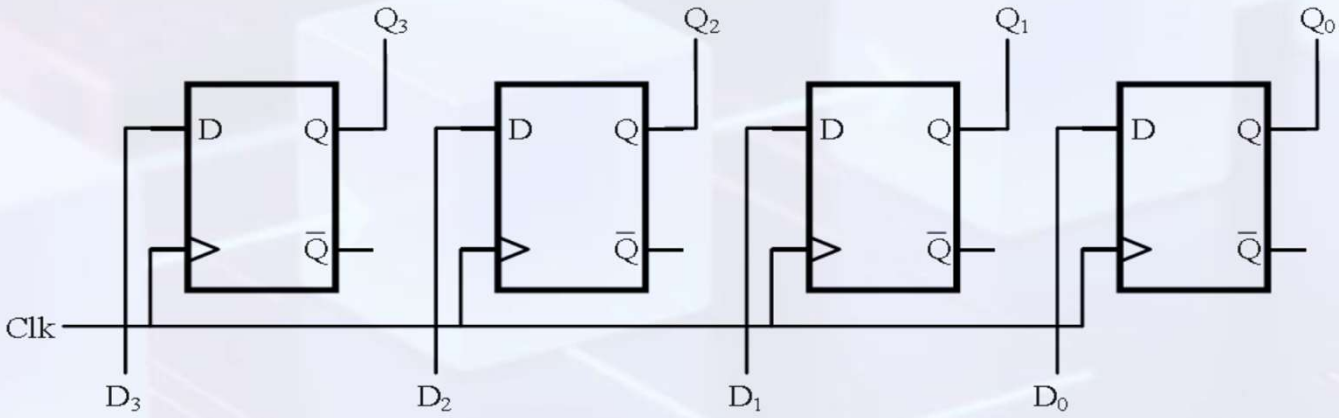
J	K	Q(t + 1)	
0	0	Q(t)	SR flip-flop
0	1	0	
1	0	1	
1	1	$\overline{Q}(t)$	T flip-flop



基本存储电路

Registers 寄存器

- Register: **n** flip-flops used to store **n** bits of information
- Use the same **Clk** signal to retain/update synchronously



基本存储电路

Shift Register 移位寄存器

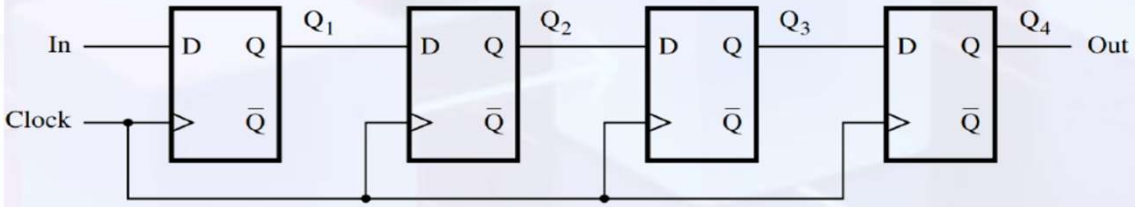
- A register that can shift its contents, e.g. used in "×2" operation

$(10)_{10}$ = 1 0 1 0

$(20)_{10}$ = 1 0 1 0 0

$(40)_{10}$ = 1 0 1 0 0 0

0 0 0 1 1 1 0 1

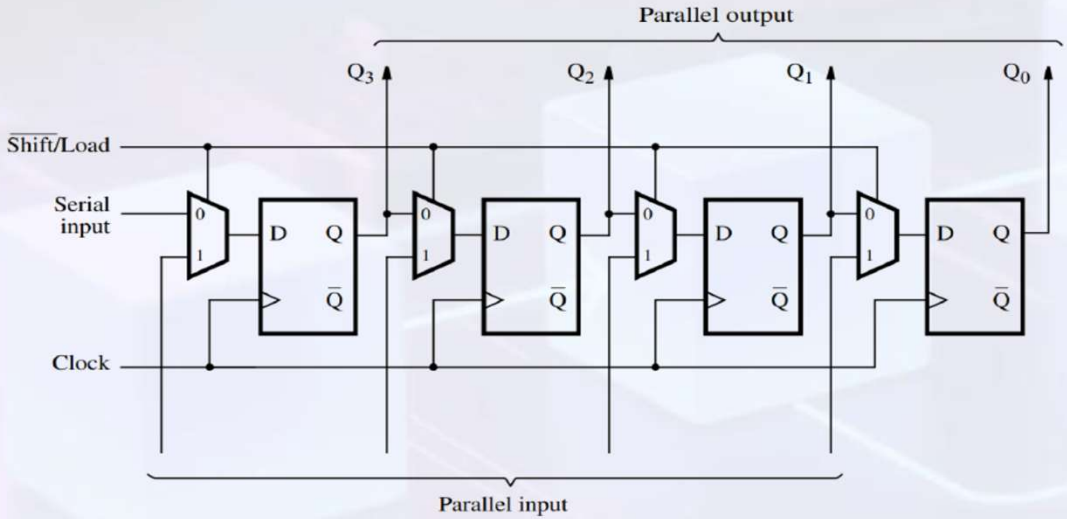


	In	Q_1	Q_2	Q_3	Q_4 = Out
t_0	1	0	0	0	0
t_1	0	1	0	0	0
t_2	1	0	1	0	0
t_3	1	1	0	1	0
t_4	1	1	1	0	1
t_5	0	1	1	1	0
t_6	0	0	1	1	1
t_7	0	0	0	1	1

基本存储电路

Shift Register with Parallel Load

- Useful in parallel-to-serial transfer 并-串转换



- If *Shift/Load* = 0, it's a shift register
- If *Shift/Load* = 1, the parallel input data are loaded into the register

Question: serial-to-parallel transfer 串-并转换 ?

基本存储电路

Counters 计数器

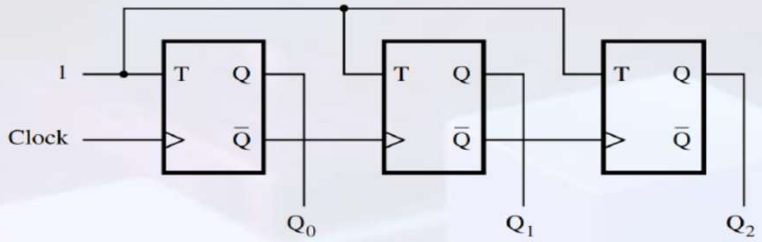
Decimal	Binary			
	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

- All bits are toggling at different frequency
- Q_0 toggles every step
- Q_1 toggles every time when Q_0 has a negative edge
- Q_2 toggles every time when Q_1 has a negative edge
- Q_3 toggles every time when Q_2 has a negative edge

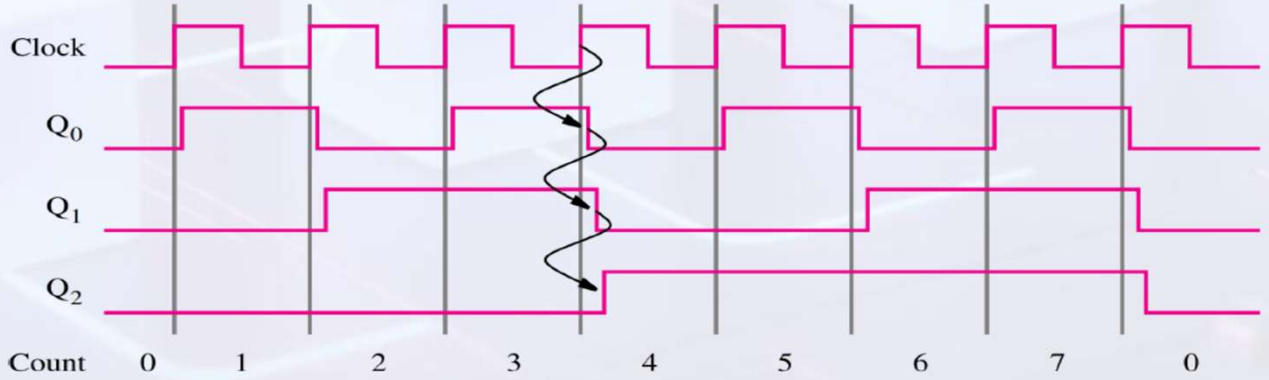
基本存储电路

Counters 计数器

- A three-bit counter with T flip-flops



- The T input of each flip-flop is connected to a constant "1", states toggling at each positive edge of clock
- Two flip-flops have their clock driven by the $\overline{Q}$ of the preceding flip-flop
- It's a up-counter 递增计数器



基本存储电路

Practice:

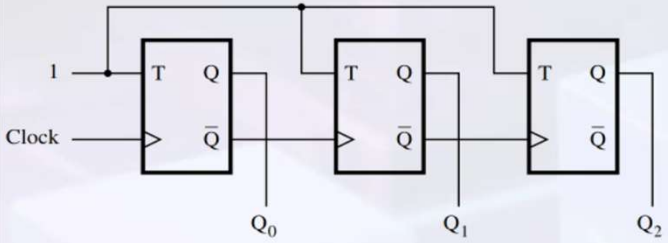
Problem: 1. How to build a down-counter 递减计数器?

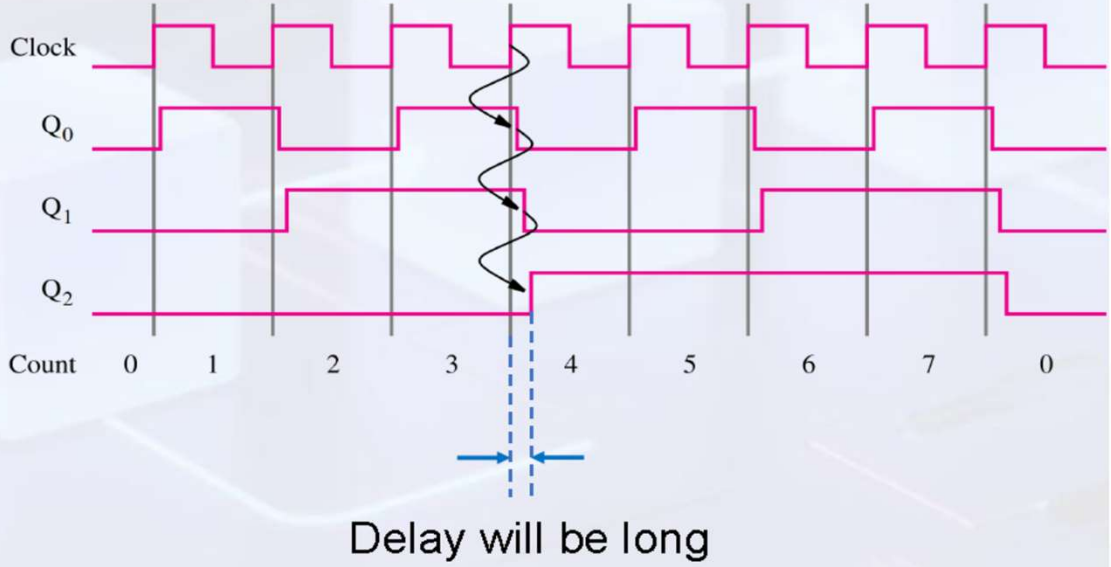
2. How to build a up/down counter ?

基本存储电路

Issue: delay due to cascaded clocking

- For a counter with a larger number of bits, the delay is long





基本存储电路

Synchronous Counters

Decimal	Binary			
	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

- Only refer to previous state
- Observation: a given flip-flop changes its state only when all the preceding flip-flops are in the state Q = 1

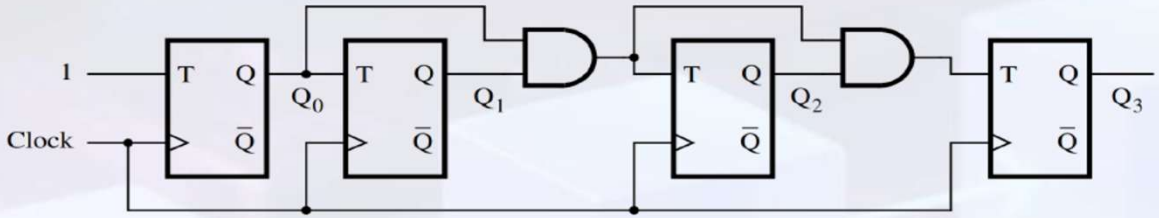
Take this as the condition for toggling:

$$T_0 = 1$$
$$T_1 = Q_0$$
$$T_2 = Q_0 Q_1$$
$$T_3 = Q_0 Q_1 Q_2$$
$$\cdots$$
$$T_n = Q_0 Q_1 \cdots Q_{n-1}$$

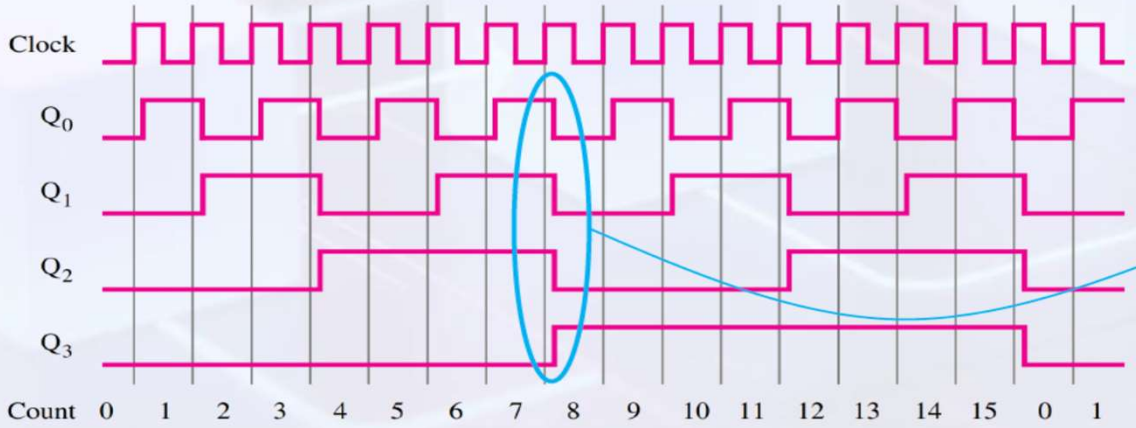
基本存储电路

Synchronous Counters

- With T Flip-Flops



$$T_0 = 1$$
$$T_1 = Q_0$$
$$T_2 = Q_0 Q_1$$
$$T_3 = Q_0 Q_1 Q_2$$

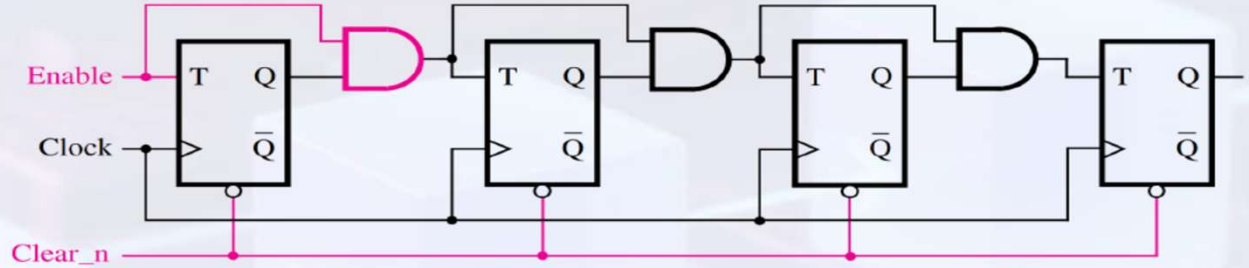


same delay for all bits

基本存储电路

Synchronous Counters

- Enable (使能) and Clear (清零)



Enable: if Enable = 0, all T inputs will be equal to 0
Clear: allows starting counting from "0"

Question: is the clear operation synchronous ?

基本存储电路

Synchronous Counters

- With D flip-flops

$$D_0 = Q_0 \oplus Enable$$

$$D_1 = Q_1 \oplus (Q_0 \cdot Enable)$$

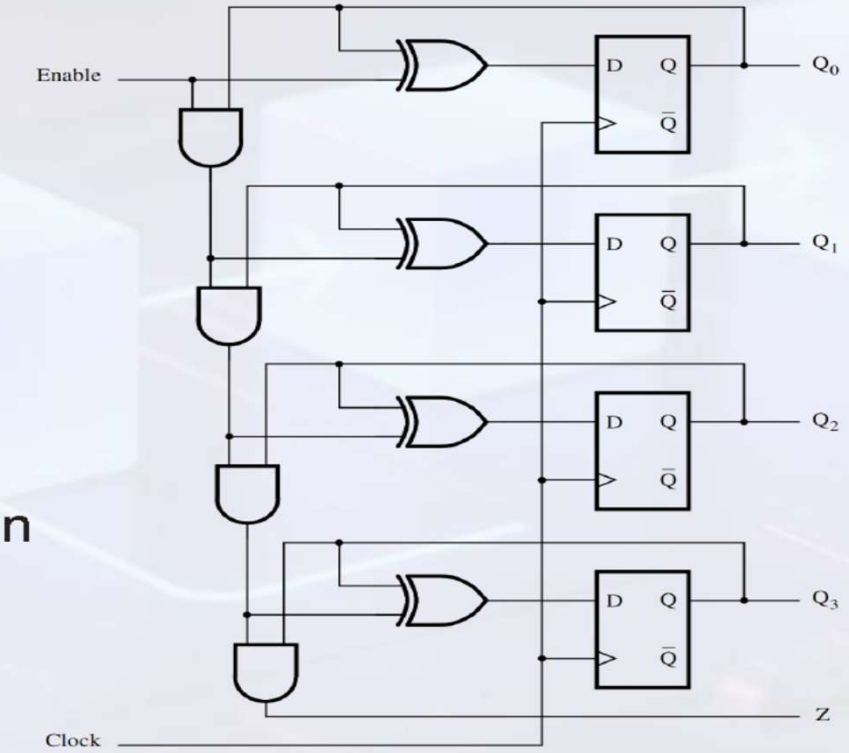
$$D_2 = Q_2 \oplus (Q_1 Q_0 \cdot Enable)$$

$$D_3 = Q_3 \oplus (Q_2 Q_1 Q_0 \cdot Enable)$$

$$\cdots$$

$$D_i = Q_i \oplus (Q_{i-1} Q_{i-2} \cdots Q_1 Q_0 \cdot Enable)$$

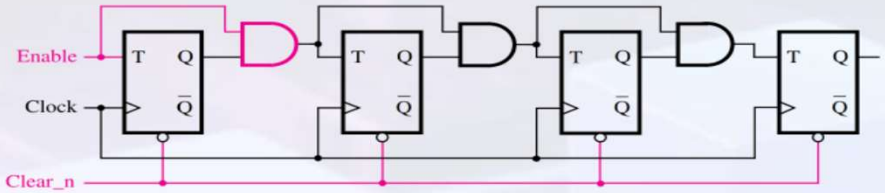
The derivation will be given in following chapters



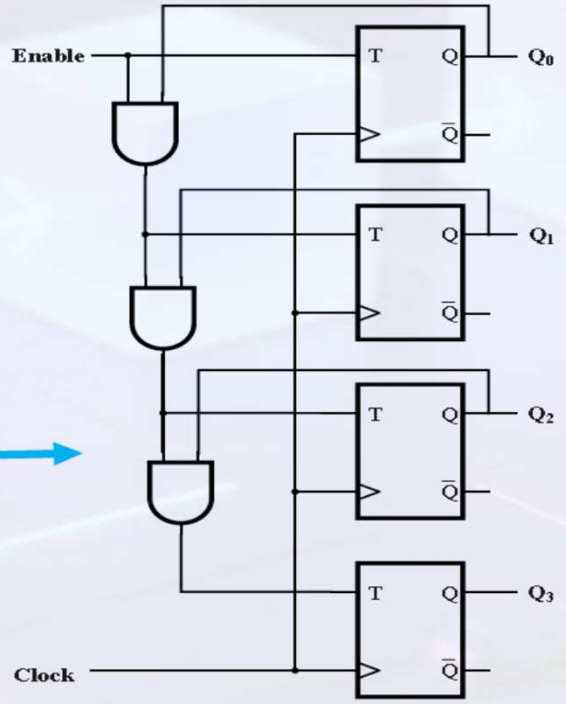
基本存储电路

Synchronous Counters

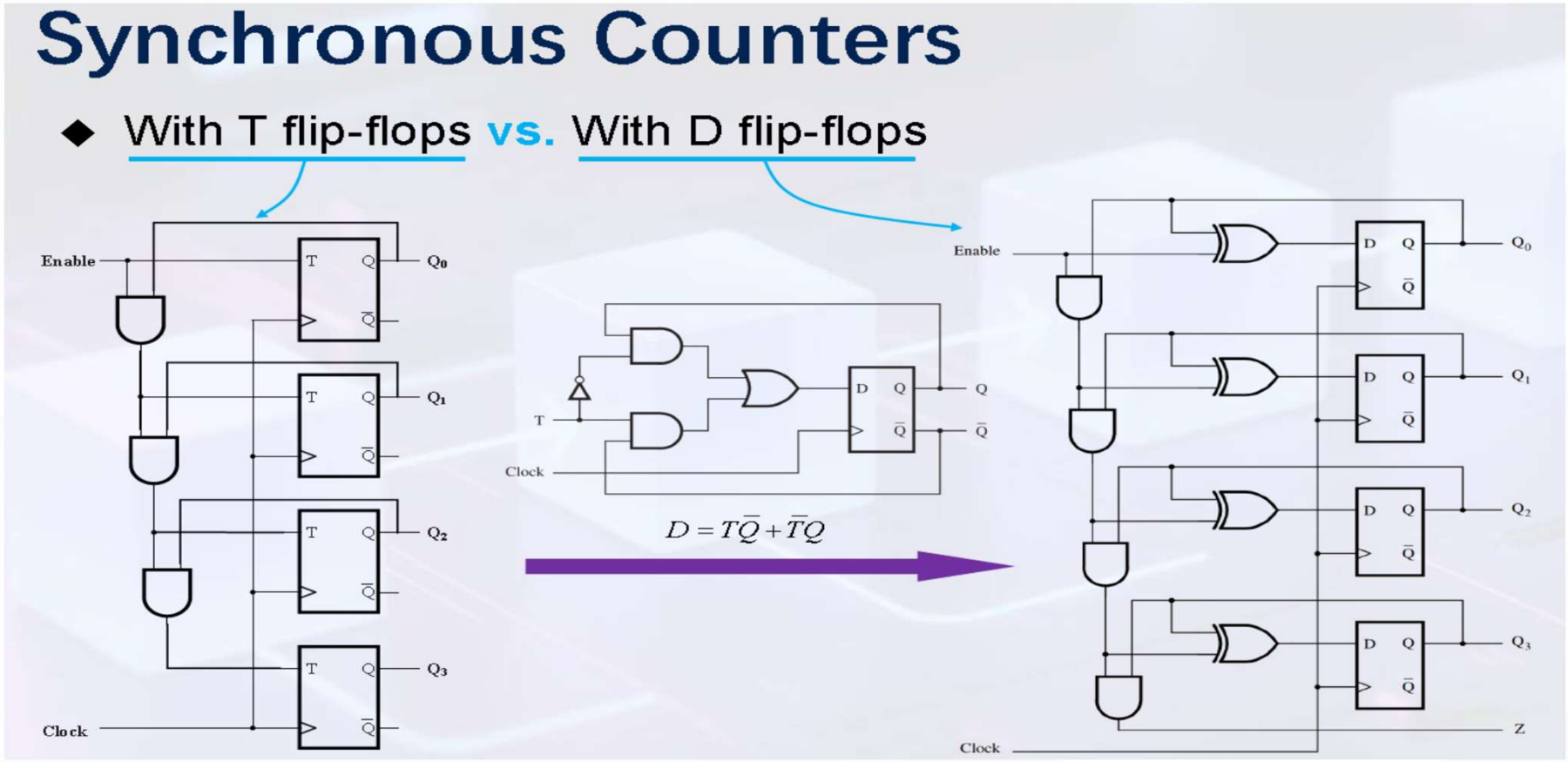
- With T flip-flops **vs.** With D flip-flops



Re-draw the one with T flip-flop
Ignore clear_n



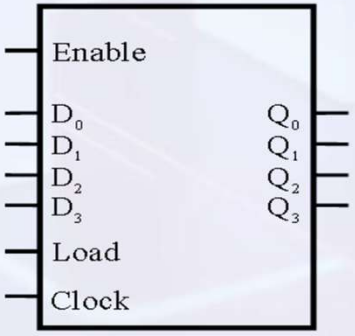
基本存储电路

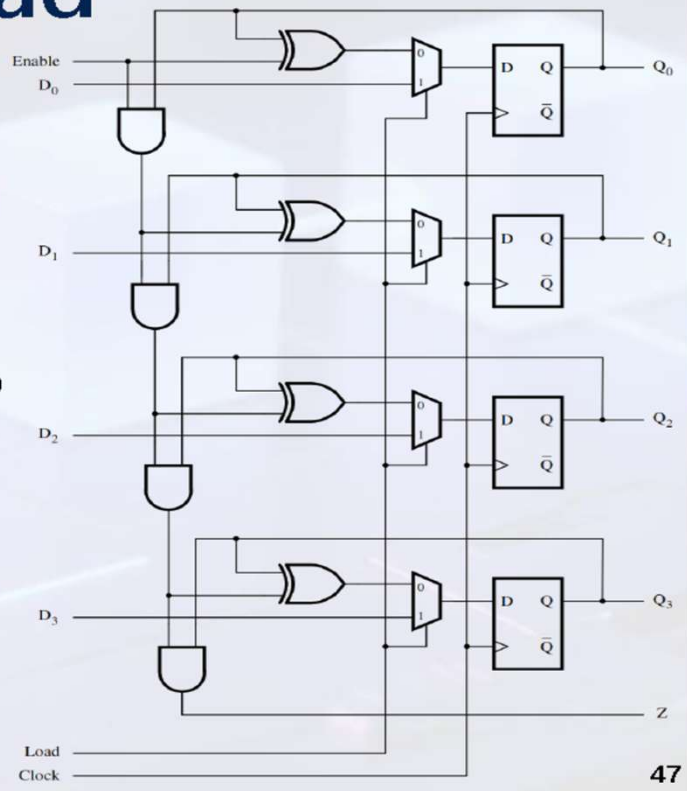

Synchronous Counters
With T flip-flops vs. With D flip-flops
Enable
T
Q
Q_0
Q_1
Q_2
Q_3
Clock
D
$D = T\bar{Q} + \bar{T}Q$
Z

基本存储电路

Counters with Parallel Load

- Sometimes it is desirable to **start with a non-zero count**
- Multiplexers controlled by "Load" are used to switch between two states:
  - Load=0: normal counting
  - Load=1: load new initial value $D_3D_2D_1D_0$ into the counter
- Symbol:



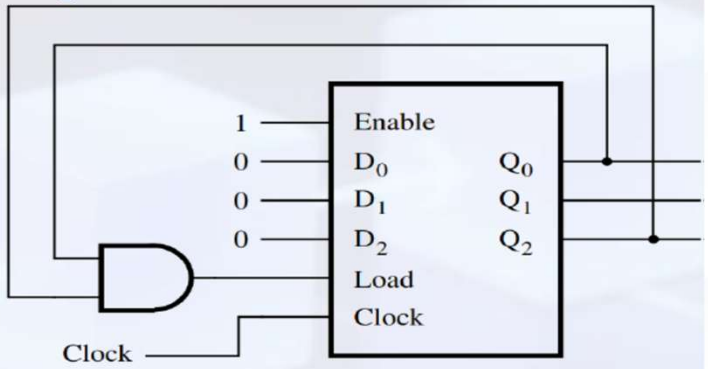


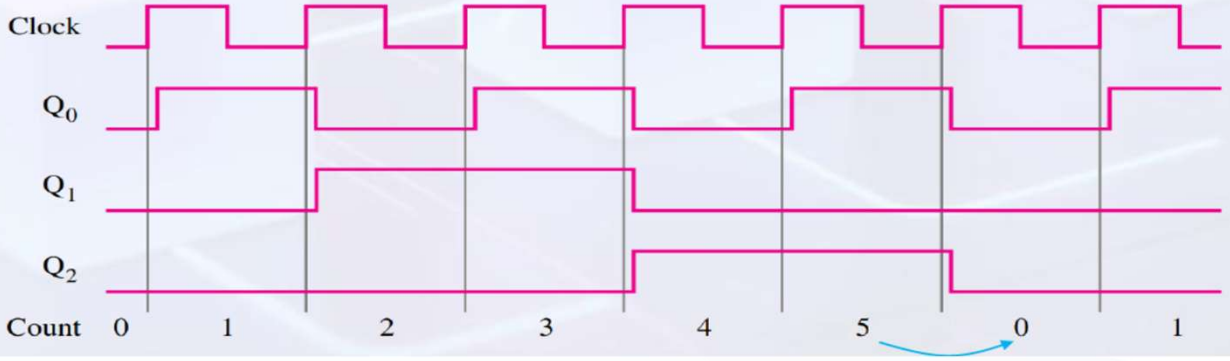
基本存储电路

Counters with Reset (复位)

- Sometimes it is desirable to **end at a certain count**
- e.g. 0 1 2 3 4 5 0 1 2 3 4 5 .....

modulo-6 counter 模6计数器



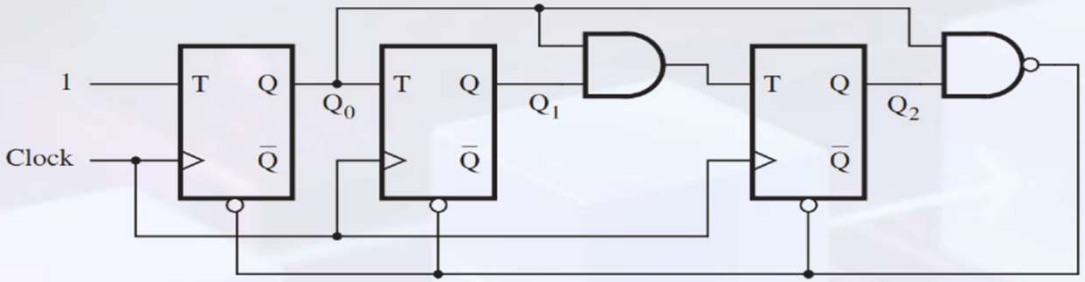


synchronous reset

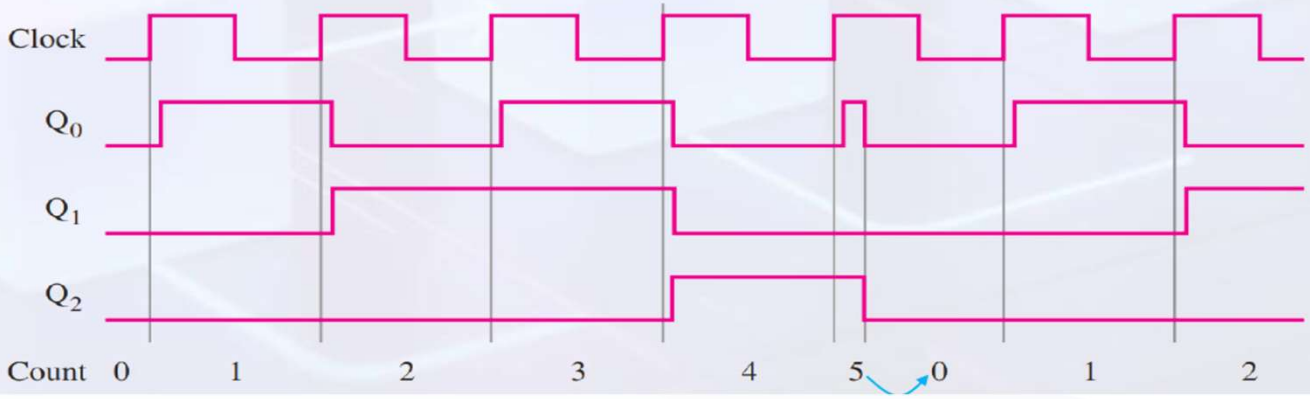
基本存储电路

Counters with Reset (复位)

- modulo-6 counter: 0 1 2 3 4 5 0 1 2 3 4 5 ……



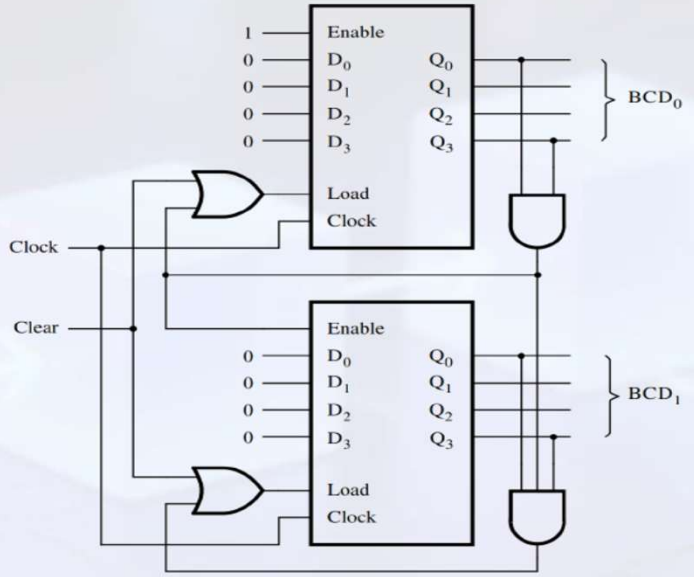
asynchronous reset



基本存储电路

BCD Counter

Count	Tens digit 十位	Units digit 个位	BCD_1				BCD_2			
			Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	1
2	0	2	0	0	0	0	0	0	1	0
3	0	3	0	0	0	0	0	0	1	1
4	0	4	0	0	0	0	0	1	0	0
5	0	5	0	0	0	0	0	1	0	1
6	0	6	0	0	0	0	0	1	1	0
7	0	7	0	0	0	0	0	1	1	1
8	0	8	0	0	0	0	1	0	0	0
9	0	9	0	0	0	0	1	0	0	1
10	1	0	0	0	0	1	0	0	0	0
11	1	1	0	0	0	1	0	0	0	1
12	1	2	0	0	0	1	0	0	1	0
13	1	3	0	0	0	1	0	0	1	1
14	1	4	0	0	0	1	0	1	0	0
15	1	5	0	0	0	1	0	1	0	1



- **1001** in units digit leads to reset of units digit and carry to tens digit
- **1001** in both digit leads to reset of tens digit

基本存储电路

Ring Counter & Johnson Counter

Ring counter 环形计数器				Johnson Counter 扭环形计数器			
Q_0	Q_1	Q_2	Q_3	Q_0	Q_1	Q_2	Q_3
1	0	0	0	0	0	0	0
0	1	0	0	1	0	0	0
0	0	1	0	1	1	0	0
0	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1
0	1	0	0	0	1	1	1
0	0	1	0	0	0	1	1
0	0	0	1	0	0	0	1
1	0	0	0	0	0	0	0
……				……			

4 states per period

8 states per period

基本存储电路

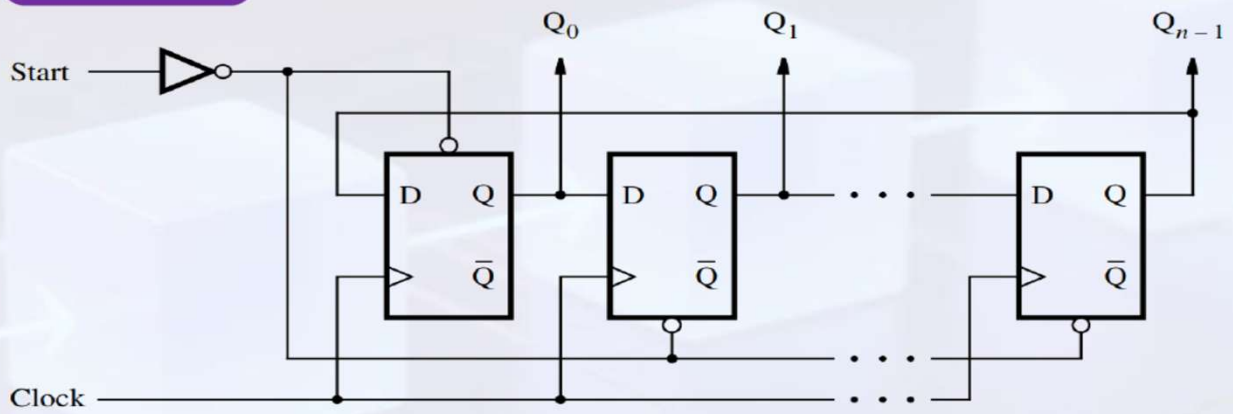
Ring Counter

Ring counter
环形计数器

Q_0	Q_1	Q_2	Q_3
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

……

Method 1



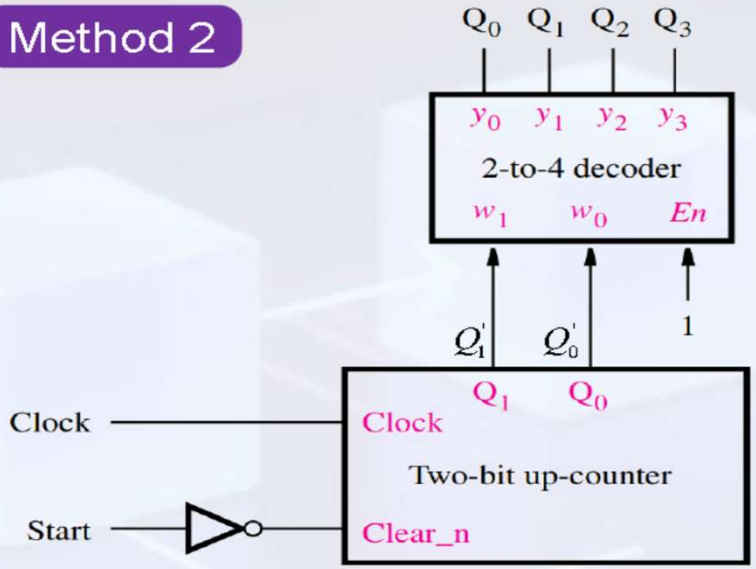
- *Start* signal presets the Q_0 to 1 and clears the others to 0
- Initialize the counter by injecting a 1 into the first stage

基本存储电路

Ring Counter

2-bit counter		decoder			
Q_1'	Q_0'	Q_0	Q_1	Q_2	Q_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1
0	0	1	0	0	0
……		……			

Method 2



- 2-to-4 decoder changes the counter output into a one-hot code

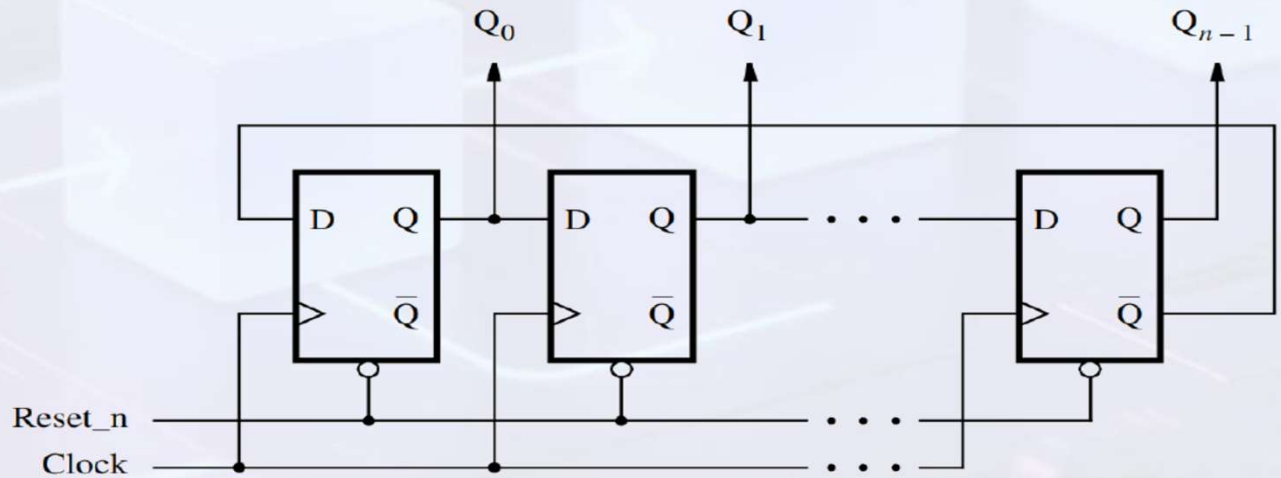
基本存储电路

Johnson Counter

Johnson Counter
扭环形计数器

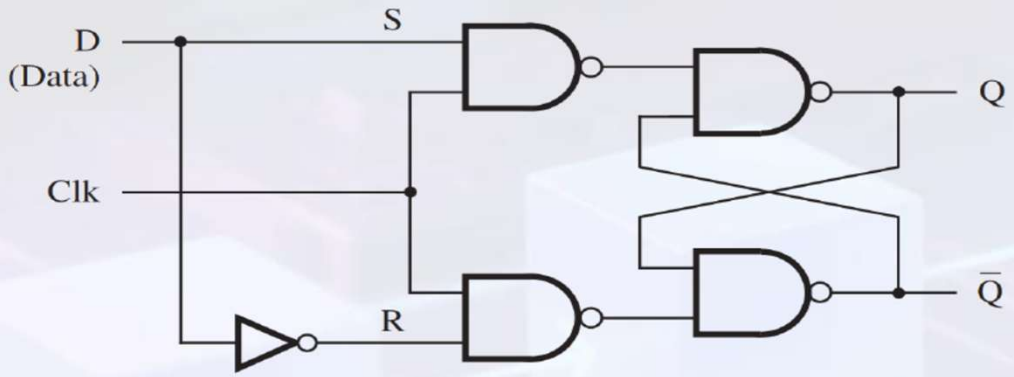
Q_0	Q_1	Q_2	Q_3	
0	0	0	0	$\overline{Q}_3$
1	0	0	0	
1	1	0	0	
1	1	1	0	
1	1	1	1	$\overline{Q}_3$
0	1	1	1	
0	0	1	1	
0	0	0	1	
0	0	0	0	
……				

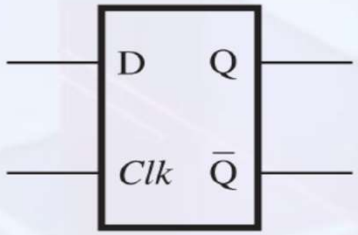
- $\overline{Q}$ output of the last stage is fed back to the first stage
- Reset signal initializes the counter into "0000"



基本存储电路

Verilog Code - Gated D Latch





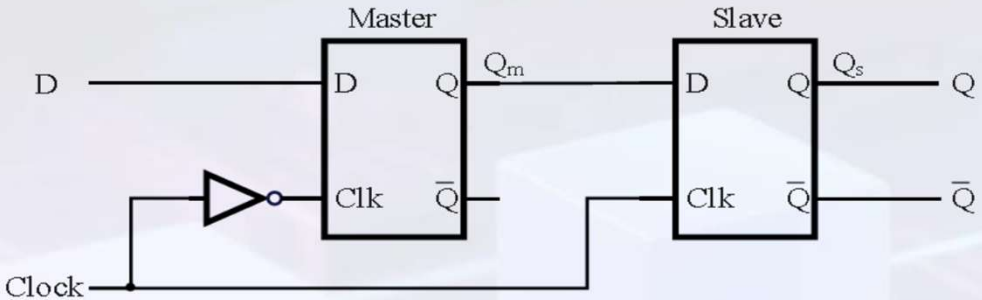

```
module D_latch (D, Clk, Q);
    input D, Clk;
    output reg Q;

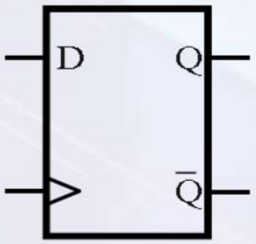
    always @(D, Clk)
        if (Clk)
            Q = D;
endmodule
```

When the action for "Clk≠1" is not assigned, Verilog compiler assumes that the value of *Q* must be maintained

基本存储电路

Verilog Code - D Flip-Flop






```
module flipflop (D, Clock, Q);
    input D, Clock;
    output reg Q;

    always @(posedge Clock)
        Q = D;

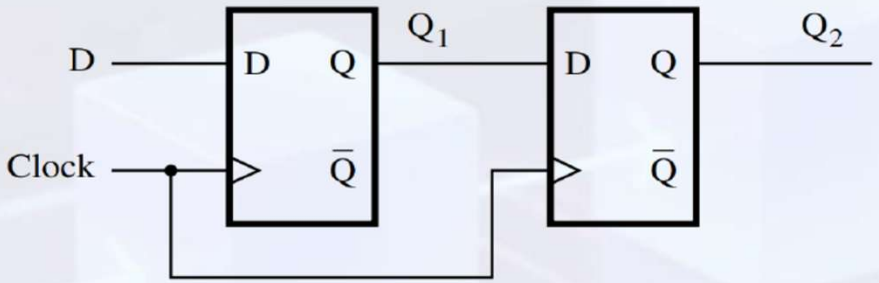
endmodule
```

posedge: positive edge of Clock
negedge: negative edge of Clock

基本存储电路

Blocking and Non-Blocking Assignments

- How will you describe the circuit below in Verilog ?

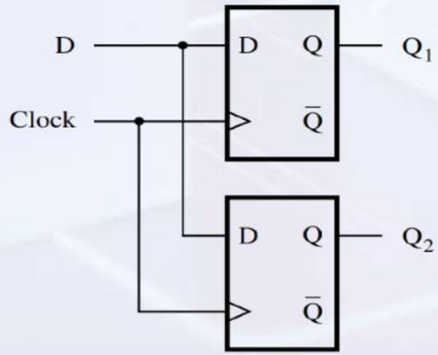


基本存储电路

Blocking and Non-Blocking Assignments

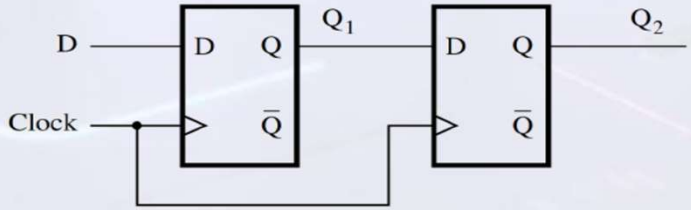
- Blocking assignment 阻塞赋值

```
module example1 (D, Clock, Q1, Q2);
    input D, Clock;
    output reg Q1, Q2;
    always @(posedge Clock)
    begin
        Q1 = D;
        Q2 = Q1;
    end
endmodule
```



- Non-blocking assignment 非阻塞赋值

```
module example2 (D, Clock, Q1, Q2);
    input D, Clock;
    output reg Q1, Q2;
    always @(posedge Clock)
    begin
        Q1 <= D;
        Q2 <= Q1;
    end
endmodule
```



基本存储电路

Blocking and Non-Blocking Assignments

- Blocking assignment 阻塞赋值

```
always @(posedge Clock)
begin
    Q1 = D;
    Q2 = Q1;
end
```

- Blocking assignment statements in an **always** block are evaluated in the order in which they are written
- If a variable is given a value by a blocking assignment statement, then this new value is used in evaluating all subsequent statements in the block

- Non-blocking assignment 非阻塞赋值

```
always @(posedge Clock)
begin
    Q1 <= D;
    Q2 <= Q1;
end
```

- Non-blocking assignment statements in an **always** block are evaluated using the values when the always block is entered
- A given variable has the same value for all statements in the block

基本存储电路

Blocking and Non-Blocking Assignments

- Blocking assignment 阻塞赋值

```
module example1 (x1, x2, x3, Clock, f, g);
    input x1, x2, x3, Clock;
    output reg f, g;
    always @(posedge Clock)
    begin
        f = x1 & x2;
        g = f | x3;
    end
endmodule
```



- Non-blocking assignment 非阻塞赋值

```
module example2 (x1, x2, x3, Clock, f, g);
    input x1, x2, x3, Clock;
    output reg f, g;
    always @(posedge Clock)
    begin
        f <= x1 & x2;
        g <= f | x3;
    end
endmodule
```



基本存储电路

Blocking and Non-Blocking Assignments

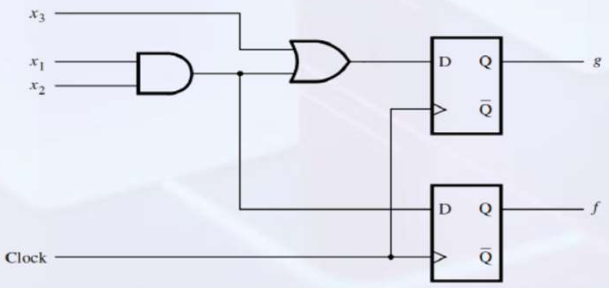
- Blocking assignment 阻塞赋值

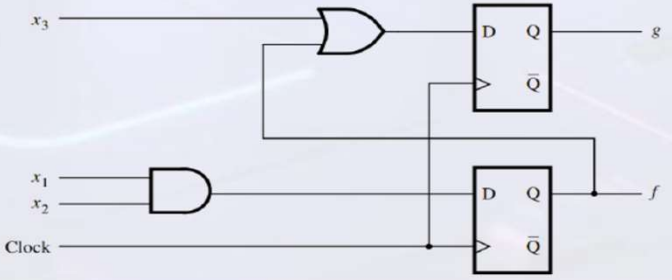
```
module example1 (x1, x2, x3, Clock, f, g);
    input x1, x2, x3, Clock;
    output reg f, g;
    always @(posedge Clock)
    begin
        f = x1 & x2;
        g = f | x3;
    end
endmodule
```

- Non-blocking assignment 非阻塞赋值

```
module example2 (x1, x2, x3, Clock, f, g);
    input x1, x2, x3, Clock;
    output reg f, g;
    always @(posedge Clock)
    begin
        f <= x1 & x2;
        g <= f | x3;
    end
endmodule
```

sequential circuit





基本存储电路

Blocking and Non-Blocking Assignments

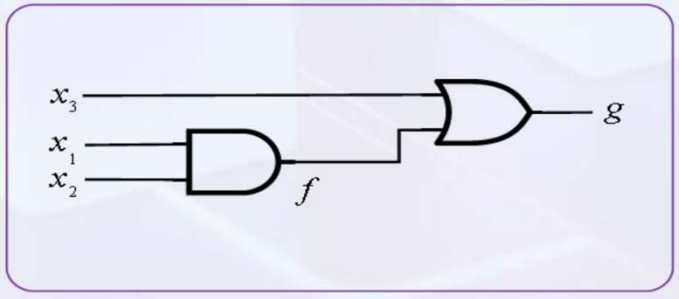
- Blocking assignment 阻塞赋值

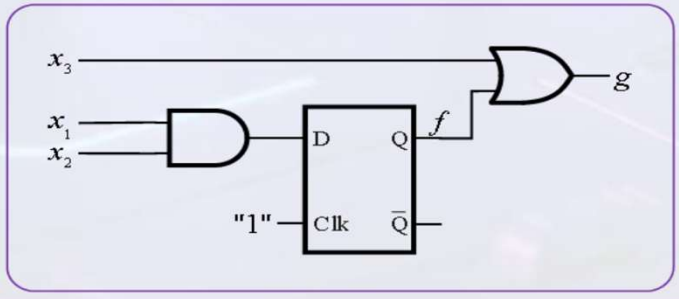
```
module example1 (x1, x2, x3, f, g);
    input x1, x2, x3;
    output reg f, g;
    always @(x1, x2, x3)
    begin
        f = x1 & x2;
        g = f | x3;
    end
endmodule
```

- Non-blocking assignment 非阻塞赋值

```
module example2 (x1, x2, x3, f, g);
    input x1, x2, x3;
    output reg f, g;
    always @(x1, x2, x3)
    begin
        f <= x1 & x2;
        g <= f | x3;
    end
endmodule
```

combinational circuit

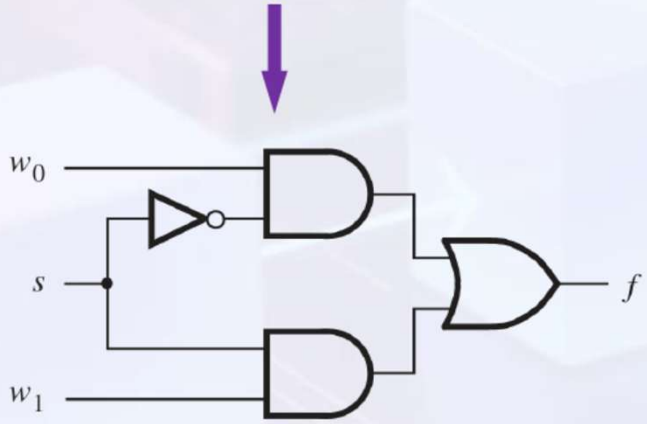




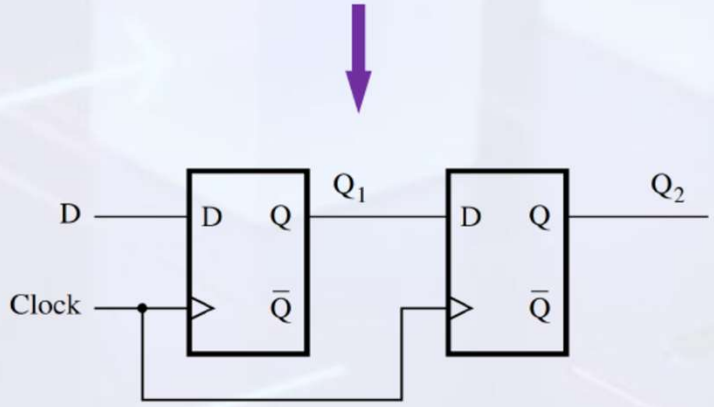
基本存储电路

Blocking and Non-Blocking Assignments

- Better use blocking assignments for combinational circuits



- Better use non-blocking assignments for sequential circuits



基本存储电路

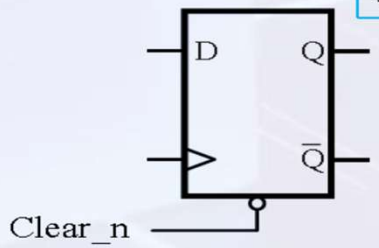
Verilog Code – DFF with Clear

```
module flipflop (D, Clock, Clearn, Q);
    input D, Clock, Clearn;
    output reg Q;

    always @(negedge Clearn, posedge Clock)
        if (! Clearn)
            Q <= 0;
        else
            Q <= D;

endmodule
```

The sensitivity list cannot have both edge-triggered and level sensitive signals
不能同时包含边沿触发和电平敏感信号

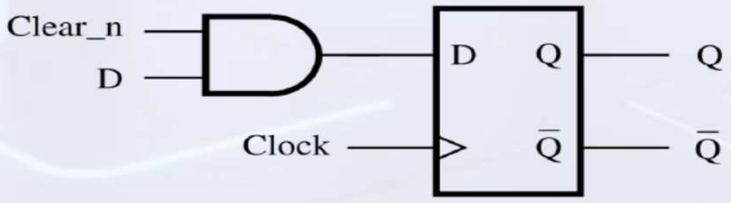


asynchronous clear

```
module flipflop (D, Clock, Clearn, Q);
    input D, Clock, Clearn;
    output reg Q;

    always @(posedge Clock)
        if (! Clearn)
            Q <= 0;
        else
            Q <= D;

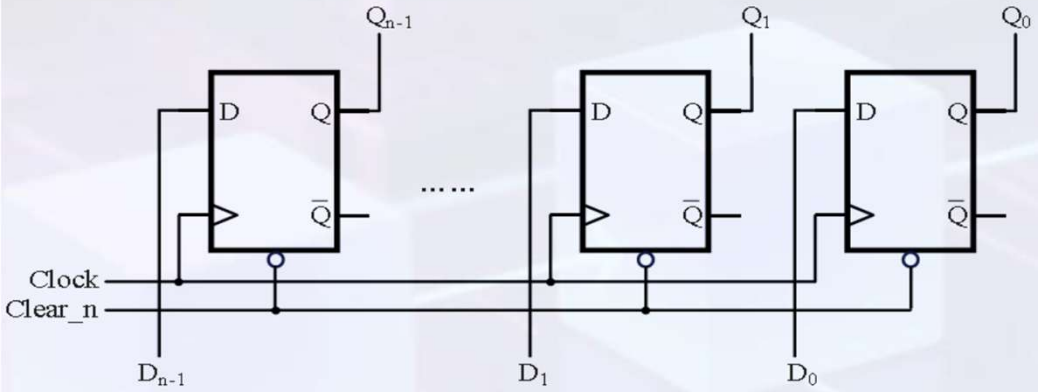
endmodule
```



synchronous clear

基本存储电路

Verilog Code - Registers with Clear

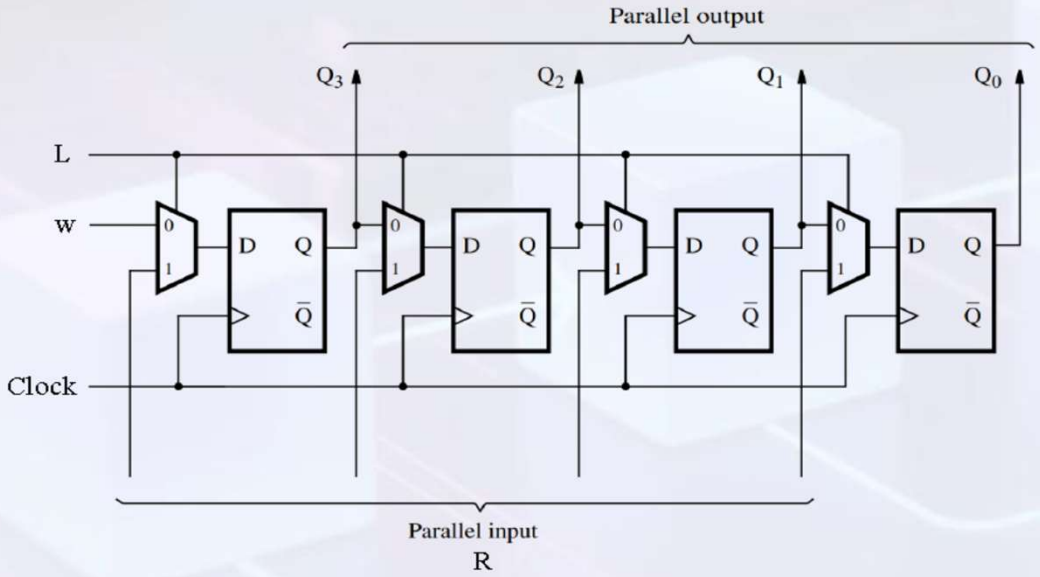



```
module regn (D, Clock, Clearn, Q);
    parameter n = 16;
    input [n –1:0] D;
    input Clock, Clearn;
    output reg [n –1:0] Q;
    always @(negedge Clearn, posedge Clock)
        if (! Clearn)
            Q < = 0;
        else
            Q < = D;
endmodule
```

- n bits
- With asynchronous clear

基本存储电路

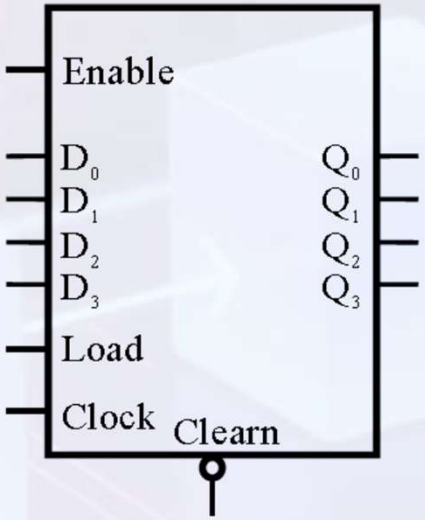
Verilog Code - Shift Register with Load




```
module shift4 (R, L, w, Clock, Q);
    input [3:0] R;
    input L, w, Clock;
    output reg [3:0] Q;
    always @(posedge Clock)
        if (L)
            Q <= R;
        else
            begin
                Q[0] <= Q[1];
                Q[1] <= Q[2];
                Q[2] <= Q[3];
                Q[3] <= w;
            end
endmodule
```

基本存储电路

Verilog Code - Counter with Clear|Load|Enable




```
module upcount (D, Clearn, Clock, E, L, Q);
    input [3:0] D;
    input Clearn, Clock, E, L;
    output reg [3:0] Q;

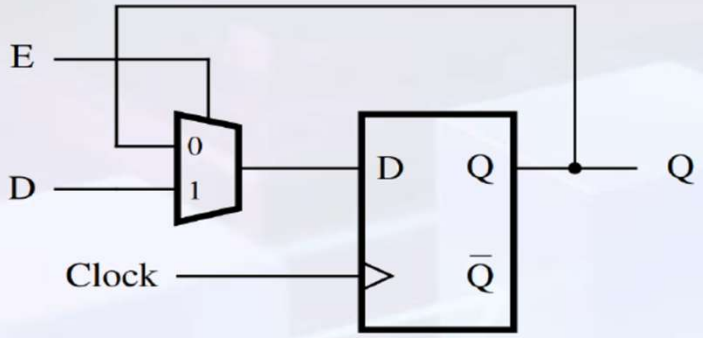
    always @(negedge Clearn, posedge Clock)
        if (! Clearn)
            Q <= 0;
        else if (L)
            Q <= D;
        else if (E)
            Q <= Q + 1;

endmodule
```

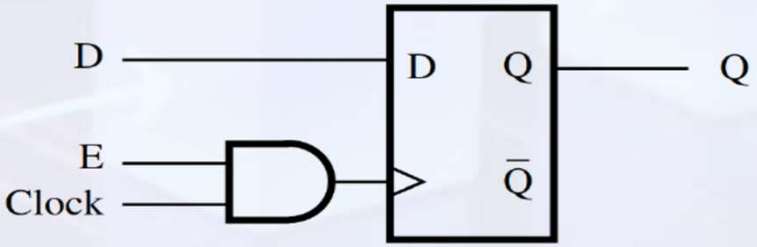
基本存储电路

Verilog Code - DFF with Enable

- Two possible ways to add “Enable”



using a multiplexer



clock gating

Question: which solution is better ?